TextAE: A Benchmark and Flow-Matching Baseline for Text-Controlled Layered Animation

Jiarong Rong*
m202572289@hust.edu.cn
Huazhong University of Science and Technology
School of Cyber Science and Engineering
Wuhan, China

Hao Yang*
charles.yh@antgroup.com
Ant Group
Hangzhou, China

Xinrui Shan
u202211905@hust.edu.cn
Huazhong University of Science and Technology
School of Cyber Science and Engineering
Wuhan, China

Di Yuan
ydmkdzir@gmail.com
Huazhong University of Science and Technology
School of Cyber Science and Engineering
Wuhan, China

Qiankun Zhang✉
qiankun@hust.edu.cn
Huazhong University of Science and Technology
School of Cyber Science and Engineering
Wuhan, China

Xianjun Deng
dengxj615@hust.edu.cn
Huazhong University of Science and Technology
School of Cyber Science and Engineering
Wuhan, China

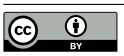
Abstract

Layered animation is a common representation in motion graphics and interface design, but authoring high-quality motion remains a manual and iteration-heavy process. Existing multimodal generation systems often target pixel-space video synthesis or full animation-program generation, making them a limited match for workflows that start from an existing visual layer and require lightweight, editable motion control. In this paper, we study *text-controlled layered animation with editable transformation tracks*: given a static image layer, a text instruction, and an initial transformation state, the goal is to generate a temporally coherent transformation track that can be further edited in professional animation tools. To support empirical study of this setting, we present *TextAE*, a benchmark built from 9,585 real animation projects and 75,520 expert-designed layer-wise transformation tracks, with a text-annotated subset of 17,774 layers. TextAE is our primary contribution and provides a foundation for studying this task. We also provide *Track-FM*, a flow-matching baseline that jointly predicts full transformation tracks from multimodal conditions while encouraging smooth and loopable motion. Experiments indicate that TextAE provides a useful testbed for controllable layered animation generation and that Track-FM serves as a strong first baseline for producing faithful, smooth, and editable animation effects.

CCS Concepts

• **Computing methodologies** → **Animation**; Neural networks; • **Information systems** → *Multimedia content creation.*

*Both authors contributed equally to the paper.
✉ Qiankun Zhang is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
MM '26, Rio de Janeiro, Brazil.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2213-4/2026/11
https://doi.org/10.1145/3767308.3835784

Keywords

layered animation generation; editable motion generation; flow matching; multimodal generation

ACM Reference Format:
Jiarong Rong, Hao Yang, Xinrui Shan, Di Yuan, Qiankun Zhang, and Xianjun Deng. 2026. TextAE: A Benchmark and Flow-Matching Baseline for Text-Controlled Layered Animation. In *Proceedings of the 34th ACM International Conference on Multimedia (MM '26), November 10–14, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3767308.3835784

1 Introduction

Animation effects are concise, parameterized transformations that bring static graphic layers to life. Rather than rendering full scenes, they describe how attributes such as position, scale, rotation, and opacity evolve over time. Typical examples include fade-in, slide-in, and spin effects. Yet designing compelling animation effects remains labor-intensive and expert-driven. In traditional workflows, designers start from existing visual layers, place keyframes on a timeline, adjust interpolation curves, preview the result, and iterate until the motion feels right. Because even small changes in timing or interpolation can substantially alter the perceived style of an effect, this process often involves repeated rounds of manual preview and refinement. The resulting motion is editable and reusable, but crafting high-quality transformation tracks still demands substantial domain knowledge and manual effort, making animation authoring costly and difficult to scale.

Recent multimodal generative models have made rapid progress in language-guided visual generation, especially text-to-video generation [7, 8, 14, 33, 35, 41, 43, 44]. However, these systems mainly output rendered pixels, which are difficult to revise at the level of motion attributes once generated. Vector-animation systems such as OmniLottie [48] move closer to structured animation generation, but they target whole animation programs rather than the more common workflow in which a designer already has a visual layer

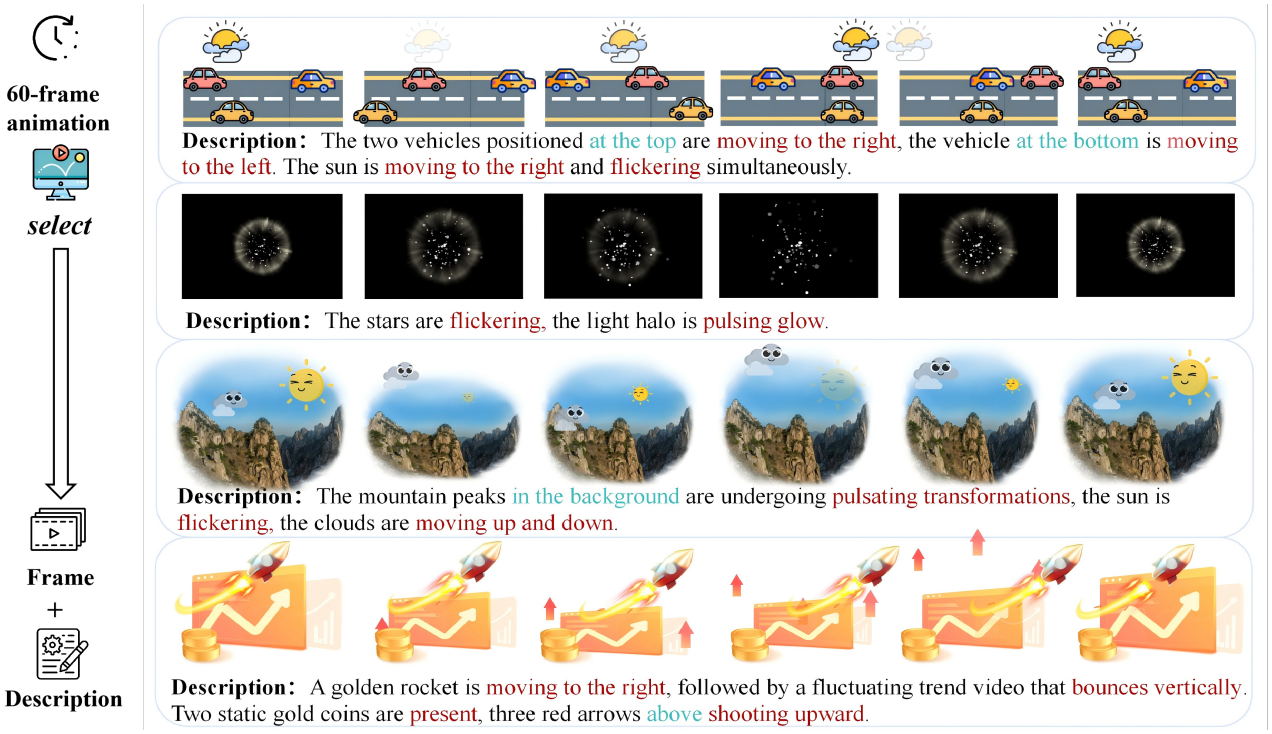


Figure 1: Overview of text-controlled layered animation. From a static layer, a text prompt, and an initial transform state, the model generates an editable transformation track for layered animation authoring. Six frames are shown from each 60-frame sequence.

and needs controllable motion for that layer. In practice, many animation workflows require a narrower setting: generating editable motion control for existing layered assets rather than synthesizing final pixels or complete programs. The goal is not to invent the visual content itself, but to produce motion controls that remain interpretable, reusable, and editable after generation.

Motivated by this gap, we study *text-controlled layered animation with editable transformation tracks*. Given a static image layer, a text instruction, and the layer's initial transformation state, the goal is to generate a layer-wise transformation track that can be rendered and further edited downstream. This problem sits between pixel-space video synthesis and full animation-program generation, focusing on reusable motion control for existing assets. It also introduces distinct technical challenges, since the desired motion depends on the layer's initial state, text often affects only a subset of transformation attributes, and the output must remain temporally smooth and directly editable. This setting remains underexplored and lacks a dedicated benchmark and a task-specific baseline.

To support empirical study of this setting, we introduce *TextAE*, a benchmark built from 9,585 real animation projects and 75,520 expert-designed layer-wise transformation tracks, among which 17,774 layers are additionally annotated with textual descriptions. TextAE preserves not only the target tracks but also the underlying layered assets and reconstructable project representations. By retaining real project structure, it reflects the practical constraints of animation authoring. It therefore supports both large-scale track supervision across all layers and text-conditioned evaluation on the annotated subset while remaining compatible with downstream editing workflows.

As a concrete baseline, we introduce *Track-FM*, a DiT-based flow-matching framework for multimodal transformation-track generation. Since tracks are continuous structured trajectories, flow matching is a natural fit. Track-FM predicts full tracks jointly from the source layer, text prompt, and initial state while encouraging smooth motion. It provides a concrete starting point for text-conditioned editable track generation in this setting.

Figure 1 presents an overview of the task. Together, TextAE and Track-FM establish a practical framework for studying controllable layered animation generation on its own terms, rather than as a by-product of broader video or program generation.

In summary, our contributions are as follows:

- We formulate text-controlled layered animation as a distinct task: generating editable layer-wise transformation tracks from an existing visual layer, a text instruction, and an initial transformation state.
- We introduce TextAE, a benchmark built from 9,585 real animation projects, with 75,520 expert-designed transformation tracks and a text-annotated subset of 17,774 layers, supporting both large-scale track supervision and text-conditioned evaluation.
- We present Track-FM, a flow-matching framework for multimodal transformation-track generation that predicts full

layer-wise trajectories from image, text, and initial-state conditions.
- We provide a comprehensive empirical evaluation and characterize the trade-offs among alignment, temporal quality, diversity, and practical pipeline reliability.

2 Related Work

We discuss related work from three perspectives: controllable visual generation, vector and animation-program generation, and structured temporal modeling for continuous sequences.

Controllable Visual Generation. Recent multimodal generation research has made rapid progress in text-to-video, image-to-video, and controllable video synthesis. Representative methods such as Make-A-Video [41], Imagen Video [37], LVDM [14], ModelScope [44], VideoCrafter [5], AnimateDiff [12], and Text2Video-Zero [18] generate temporally coherent visual content from text or visual inputs. A related line studies more explicit motion control through trajectories, clicks, masks, or dragging signals, including sparse-trajectory control [13], Click-to-Move [2], Trailblazer [26], DragNUWA [50], MotionCtrl [45], Tora [53], and several recent drag- and mask-based methods [4, 19, 22, 30, 40, 46, 49, 54]. More recent video generation models such as Seedance 2.0 [39] push the frontier of general video synthesis, while architectures like Matten [9] and M4V [17] explore efficient attention and Mamba-based alternatives for spatiotemporal modeling. Specialized controllable generation systems such as MagicFight [16] further demonstrate the growing interest in fine-grained motion control. Despite stronger control, these methods still output rendered pixels, making them a limited fit for workflows that require editable layer-wise motion parameters rather than finished videos.

Vector and Animation-Program Generation. A closer direction studies editable vector representations and structured animation programs. Compared with raster video, vector animation is lightweight, resolution-independent, and easier to edit in downstream design tools. Prior work on vector graphics includes learned SVG representations [25], hierarchical SVG generation [3], differentiable rasterization [20], vector synthesis without vector supervision [34], and text-guided SVG generation [47]. Recent methods such as vector-quantized diffusion [11], Kaleido Diffusion [10], text-guided cinemagraph synthesis [27], AniDoc [28], and OmniLottie [48] further demonstrate the appeal of explicit structured outputs. Among them, OmniLottie is the closest prior work, generating complete Lottie JSON programs from multimodal inputs. However, these methods target whole scenes or full programs, whereas we study a narrower setting where the visual layer already exists and the goal is to generate compact, editable transformation tracks for that layer.

Structured Temporal Modeling. Another relevant line models structured temporal signals, including trajectory forecasting, human motion generation, and continuous generative modeling for sequences. Social LSTM [1], Trajectron++ [38], Human Motion Diffusion Model [42], MotionDiffuse [52], and DART [55] show that when the target is a low-dimensional temporal sequence rather than an image, the main challenges shift toward condition injection, temporal consistency, and sequence-level structure. Diffusion and flow-based methods [6, 15, 23, 24, 31], together with control-oriented conditioning methods such as ControlNet [51], DreamBooth [36], Null-Text Inversion [29], SDXL [32], and UPainting [21], further provide strong tools for continuous conditional generation. Our problem shares this structured-output nature but differs in semantics: we model design-oriented transformation attributes rather than physical motion. Despite these advances, the setting of generating editable layer-wise tracks from existing visual layers under text control still lacks a dedicated benchmark.

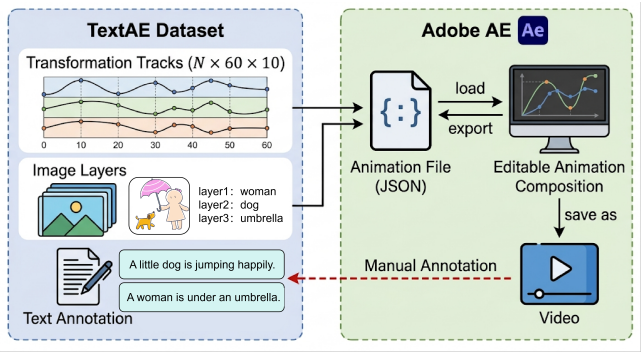


Figure 2: Workflow for reconstructing AE animations from static image layers and keyframe transformation tracks. TextAE stores each layer with its corresponding transformations, enabling editable animation generation. N denotes the number of layers.

3 TextAE Benchmark

Unlike benchmarks centered on rendered videos, TextAE is built on an editable animation representation. It models layered animation with transformation tracks derived from static image layers and initial transformation states, with textual instructions for an annotated subset. Each sample contains three elements: (1) an image layer, (2) an expert-designed transformation track, and (3) a reconstructable JSON animation file for playback or editing in Adobe After Effects (AE)[1], LottieFiles Editor[2], or Galacean Effects[3]. TextAE supports quantitative supervision and editing-oriented evaluation close to production authoring assets. This section summarizes the representation, construction pipeline, annotation protocol, quality control, and dataset statistics.

3.1 Representation and Construction

Image layers. TextAE adopts a layer-based representation. Each image layer corresponds to a visual element in a composite scene, and a complete animation is produced by applying time-varying transformations to all layers. This representation preserves editability, aligns with common production workflows, and isolates motion control from asset creation.

Transformation tracks. A transformation track describes the temporal evolution of a layer's attributes. In TextAE, each layer

[1]Adobe After Effects: https://www.adobe.com/products/aftereffects.html
[2]LottieFiles Editor: https://lottiefiles.com/lottie-editor
[3]Galacean Effects: https://www.galacean.com/effects

has 60 keyframes, and each keyframe is a 10-dimensional vector containing position, scale, rotation, and opacity. Position, scale, and rotation are parameterized along the x, y, and z axes, while opacity is a scalar. Table 1 summarizes these parameters. This track is the primary prediction target in our benchmark.

Table 1: Transformation parameters in TextAE and their effects.

Parameter	Description
Position (x_p, y_p, z_p)	Translates the object along the spatial axes.
Scale (x_s, y_s, z_s)	Resizes the object along the spatial axes.
Rotation (x_r, y_r, z_r)	Rotates the object around its local axes.
Opacity (o)	Controls the transparency of the object.

Animation reconstruction. Besides layer-wise tracks, TextAE provides a JSON animation file storing the full sequence and keyframe transformations. This file is reconstructed from layer content and tracks, and can be loaded into AE for editing or rendering, tying model outputs to the same environment used in manual authoring. Figure 2 illustrates this workflow.

Data collection. Collecting editable animation data is harder than collecting rendered videos because it requires synchronized access to layer assets, hierarchies, and keyframe tracks. We collaborated with professional designers and collected 9,585 real-world animation files across diverse scenarios.[4] Layer assets were sourced from public resources (e.g., Iconfont[5]) and designer-created materials under compatible usage conditions. We extract layer-wise tracks into JSON, yielding 9,585 animations and 75,520 layers. Our public release provides per-layer 60×10 JSON tracks and paired text annotations for a distributable subset, withholding layer images restricted by privacy or licensing constraints.

Preprocessing. Since source animations vary in duration, we standardize each transformation track to 60 keyframes. For sequences shorter than 60 frames, we repeat the last frame to fill remaining positions; for sequences longer than 60 frames, we retain the first 60 keyframes. This yields a fixed-size representation for training and evaluation with a consistent benchmark target. This standardization ensures uniform metric computation; the framework imposes no length constraint, and variable-length generation is a straightforward extension.

3.2 Annotation and Quality Control

Text annotation. We annotate a subset of layer-wise tracks using five fields: *object*, *action*, *direction*, *manner*, and *effect*. This schema is compact and expressive, with fine-grained dynamics (e.g., speed and rhythm) mainly captured by *manner* and *effect*. In total, 17,774 layers from 2,987 animations (23.5% of the full dataset) are text-annotated. These samples form the core released subset, where each instance contains a 60×10 JSON track and its paired text label. The remaining layers are retained as track data, and all 75,520 layers can be used for quantitative supervision in the full internal benchmark.

Plausibility of transformation data. After standardization, we filter samples with missing values, abnormal frame-to-frame jumps, negative scale, or opacity outside $[0, 1]$. These checks enforce structural validity and stable rendering.

Alignment between text and animation. To reduce annotation noise, we apply automatic text-motion consistency checks on value ranges, temporal continuity, dominant direction, motion intensity, and inter-attribute coherence. Motion statistics are compared with caption semantics (e.g., "slowly", "fading"), and inconsistent samples are sent for manual review.

3.3 Dataset Statistics and Split

TextAE includes 9,585 animations and 75,520 layers, of which 17,774 layers from 2,987 animations are text-annotated. All tracks are standardized to 60×10 and validated within fixed numeric ranges. The corpus covers diverse application domains: UI (34.5%), advertising (26.8%), educational (21.9%), and social-media (16.8%). The remaining 57,746 unannotated layers provide text-free motion supervision for the internal benchmark. We use strict animation-level splitting (no animation overlap across splits), then derive layer-level samples within each partition. Text-conditioned evaluation is conducted on annotated train/validation/test splits, and additional track-only supervision is limited to training animations (including non-annotated layers in the training split). For public release, we provide the annotated subset as 60×10 JSON tracks plus paired text labels; layer images are excluded when redistribution is disallowed by privacy or licensing constraints. Reproducibility is supported by fixed splits, deterministic preprocessing, and JSON-level reconstruction checks. Additional filtering, annotation, and dataset statistics are summarized in the Supplementary material. Table 2 lists the official split counts for the text-annotated subset used in text-conditioned benchmarking.

Table 2: Animation-level split and inherited layer counts for the text-annotated subset of TextAE.

Split	Animations	Layers	Annotated Layers
Train	2,390	45,192	14,220
Validation	299	5,650	1,777
Test	298	5,648	1,777
Total	2,987	56,490	17,774

4 Flow-Matching Baseline for Editable Transformation Track Generation

We present **Track-FM**, a flow-matching baseline for text-controlled layered animation with editable transformation tracks. Given an image layer, a text instruction, and the layer's initial transformation state, it predicts a full transformation track that is temporally coherent, text-aligned, and directly usable in downstream animation tools. Track-FM learns a continuous generative flow from a simple

[4] All designers contributing to TextAE have more than five years of industry experience.
[5] https://www.iconfont.cn/

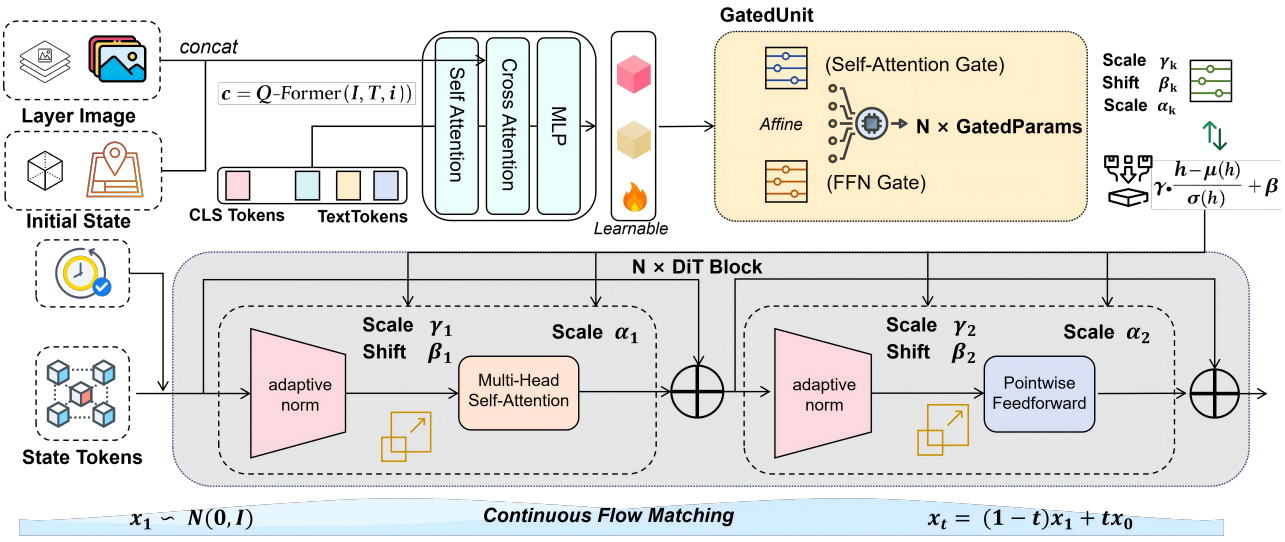


Figure 3: Model structure of Track-FM.

prior to the target track distribution. Figure 3 shows the overall architecture.

4.1 Overall Formulation

Problem setup. Given an image layer I, a textual description T, and the layer's initial transformation state $\mathbf{i}$ (including position, scale, rotation, and opacity at the first frame), Track-FM predicts a transformation track sequence

$$\mathrm{S} = G(I, T, \mathbf{i}), \tag{1}$$

where $\mathrm{S} \in \mathbb{R}^{K\times 10}$ denotes the predicted sequence with K keyframes, and each keyframe is represented by a 10-dimensional transformation vector. Track-FM operates at the layer level: for an animation with N layers, the model is applied to each layer separately, and the resulting layer-wise tracks are assembled into a scene-level JSON animation file only at export time. The initial state is essential: even under the same prompt, the desired motion depends on where the layer starts. For example, under *"burst outward,"* a left-aligned layer should move leftward, while a right-aligned layer should move rightward. This dependency cannot be resolved from text alone.

Flow-matching formulation. Instead of predicting the track autoregressively, which is prone to cumulative errors and long-horizon inconsistency, Track-FM models the entire transformation sequence jointly with continuous-time flow matching. We define a linear interpolation path between a clean data sample $\mathbf{x}_0$ and a Gaussian noise sample $\mathbf{x}_1 \sim \mathcal{N}(\mathbf{0}, I_D)$:

$$\mathbf{x}_t = (1-t)\,\mathbf{x}_1 + t\,\mathbf{x}_0, \quad t \sim \mathcal{U}(0,1), \tag{2}$$

where I_D denotes the identity covariance in D dimensions. The model learns a time-dependent velocity field

$$\mathbf{v}_\theta(\mathbf{x}_t, t, I, T, \mathbf{i}), \tag{3}$$

which transports samples along this path from the prior distribution to the target track distribution.

The corresponding flow-matching objective is

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{\mathbf{x}_0,\mathbf{x}_1,t}\left[\|\mathbf{v}_\theta(\mathbf{x}_t, t, I, T, \mathbf{i}) - (\mathbf{x}_0 - \mathbf{x}_1)\|_2^2\right]. \tag{4}$$

Compared with discrete-time noise prediction, this formulation directly optimizes the transport direction of the full trajectory and better matches the need for globally coherent motion generation.

Inference. At inference time, Track-FM generates trajectories by integrating the conditional ODE

$$\frac{\mathrm{d}\mathbf{x}_t}{\mathrm{d}t} = \mathbf{v}_\theta(\mathbf{x}_t, t, I, T, \mathbf{i}), \quad \mathbf{x}_{t=0} \sim \mathcal{N}(\mathbf{0}, I_D), \tag{5}$$

from $t = 0$ to $t = 1$, generating the full track as a coherent whole rather than as a sequence of locally optimized steps.

4.2 Multimodal Track Generator

Architecture overview. Track-FM takes three heterogeneous inputs: the image layer I, the initial state $\mathbf{i}$, and the text description T. We first encode the image layer and initial state into latent tokens, and then fuse them with text embeddings through a Q-Former-style multimodal adapter. This produces a set of joint context tokens $\mathbf{c}$ that summarize the multimodal conditions.

The context tokens condition a stack of gated DiT blocks, each of which is explicitly aware of the continuous time variable t and predicts the velocity field over the full transformation track:

$$\mathbf{v}_\theta = f_\theta(\mathbf{x}_t, t \mid \mathbf{c}). \tag{6}$$

This design enables full-track generation with global temporal context, while preserving flexible conditioning from image, text, and initial state.

Multi-control Gated Adaptive Normalization (MGAN). To incorporate multimodal control signals effectively, we use gated adaptive modulation inside each DiT block. Given the multimodal

context $\mathbf{c}$, the block predicts modulation parameters $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ for normalized hidden activations $\mathbf{h}$:

$$\text{GatedNorm}(\mathbf{h}) = \boldsymbol{\gamma}(\mathbf{c}) \odot \frac{\mathbf{h} - \mu(\mathbf{h})}{\sigma(\mathbf{h})} + \boldsymbol{\beta}(\mathbf{c}), \tag{7}$$

where $\odot$ denotes element-wise multiplication. This mechanism allows different transformation attributes, such as position, scale, rotation, and opacity, to be emphasized differently under different conditions.

Multi-condition Classifier-Free Guidance (MCFG). To improve conditional faithfulness, we apply classifier-free guidance in velocity space:

$$\mathbf{v} = \mathbf{v}_{\text{uncond}} + s\,(\mathbf{v}_{\text{cond}} - \mathbf{v}_{\text{uncond}}), \tag{8}$$

where s is the guidance scale. The unconditional branch is obtained by replacing the text prompt with an empty embedding. This design strengthens text adherence while preserving controllability from the image layer and the initial state.

4.3 Smoothness Regularization

Trajectory regularization. Since animation tracks should be visually smooth and suitable for editing, we further regularize the reconstructed clean trajectory to suppress abrupt temporal fluctuations. For the linear interpolation path, the clean trajectory estimate can be recovered from $(\mathbf{x}_t, t)$ and the predicted velocity as

$$\hat{\mathbf{x}}_0 = \mathbf{x}_t + (1-t)\,\mathbf{v}_\theta(\mathbf{x}_t, t, I, T, \mathbf{i}). \tag{9}$$

We define two lightweight regularization terms over the reconstructed sequence. The first is a second-order smoothness penalty:

$$\mathcal{L}_{\text{SR}} = \sum_{k=2}^{K-1} \left\| \hat{\mathbf{x}}_0^{k+1} - 2\hat{\mathbf{x}}_0^{k} + \hat{\mathbf{x}}_0^{k-1} \right\|_2^2. \tag{10}$$

The second is an endpoint-consistency term:

$$\mathcal{L}_{\text{loop}} = w_{\text{loop}} \left\| \hat{\mathbf{x}}_0^1 - \hat{\mathbf{x}}_0^K \right\|_2^2, \quad w_{\text{loop}} = \exp\!\left(- \left\| \mathbf{x}_0^1 - \mathbf{x}_0^K \right\|_2^2 \right), \tag{11}$$

where $\mathbf{x}_0$ is the ground-truth track. This weighting makes the endpoint penalty strong when the reference track exhibits similar start and end states and weak otherwise.

Final objective. The final training objective combines flow matching with smoothness and endpoint regularization:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{FM}} + \lambda_{\text{SR}}\,\mathcal{L}_{\text{SR}} + \lambda_{\text{loop}}\,\mathcal{L}_{\text{loop}}, \tag{12}$$

where $\lambda_{\text{SR}} \geq 0$ and $\lambda_{\text{loop}} \geq 0$ control the strengths of the two regularizers.

Summary of components. For later reference, Track-FM contains four key components: **Flow-Matching Trajectory Modeling (FTM)**, **Multi-control Gated Adaptive Normalization (MGAN)**, **Multi-condition Classifier-Free Guidance (MCFG)**, and **Smoothness Regularization (SR)**. These components correspond directly to the branch-style variants in the experiments.

5 Experiments

Unless otherwise noted, the prediction target is a layer-wise 60×10 transformation track over a fixed 2-second duration; for an animation with N layers, the exported JSON contains N such tracks.

5.1 Experimental Setup

Data Split and Sampling. We follow the official TextAE split and report results on the held-out test set. The split is defined at the animation-file level, so all layers from one animation belong to exactly one split and no test/validation animation contributes track supervision during training. For each test instance, we generate $M = 10$ samples with fixed seeds ($\{0, \ldots, 9\}$) and report averages over seeds and the test set. The main paper reports point estimates. All quantitative metrics are first computed per layer and then macro-averaged across test layers; animation-level averages are reported in the Supplementary material for consistency checks.

Compared Methods. We compare two groups of methods under a unified protocol. The first group contains methods defined under the same transformation-track interface:

- **Autoregressive**: a sequence-generation baseline that predicts transform states frame by frame.
- **DDPM + MGAN + MCFG + SR**: a denoising-diffusion baseline equipped with the same conditioning and regularization components as our final model.
- **Track-FM**[1]: the FTM-only variant, used to isolate the effect of the flow-matching trajectory formulation.
- **Track-FM**[2]: Track-FM[1] augmented with MGAN and MCFG.
- **Track-FM**[3]: Track-FM[1] augmented with SR.
- **Track-FM**: the final full model.

Track-FM[2] and Track-FM[3] are parallel branch ablations from Track-FM[1] rather than cumulative variants. The second group adapts OmniLottie [48], the closest program-level vector-animation baseline, to the same transformation-track interface. We emphasize that OmniLottie targets whole animation-program generation rather than per-layer track prediction, so this comparison serves as a feasibility reference, not a head-to-head superiority claim. We report three variants with increasingly rich condition sets: **OmniLottie-Text**, **OmniLottie-Text+Image**, and **O-mniLottie-Full**.

Comparison Protocol. For the internal comparison, all methods use matched backbone scale, training data split, and the same layer-wise output format (60×10). For the OmniLottie comparison, we use an aligned protocol with a deterministic "program-to-track" adapter: generated Lottie programs are parsed to per-layer transform channels, resampled to 60 frames, and mapped to the canonical 10 attributes. Conditions are exposed in increasing scope (text, text+image, text+image+initial state), and we keep the same split, horizon, and per-sample inference budget across methods.

Implementation Details. Compared models use aligned Transformer depth, hidden width, attention heads, and encoder family unless a method's design prevents exact matching. All tracks use a fixed 60-frame, 2-second horizon with matched guidance settings. Additional protocol details are in the Supplementary material.

The evaluation metrics are defined as follows.

Text and track alignment. It is measured by human score, LLM score, cosine similarity (CS), Pearson correlation coefficient (PCC), and dynamic time warping (DTW) against ground-truth tracks. Human score uses 30 annotators on all 1,777 test layers, with 3 independent ratings per sample under a blind and randomized protocol, using a shared 3-level rubric (0/0.5/1 for non-compliant/partially

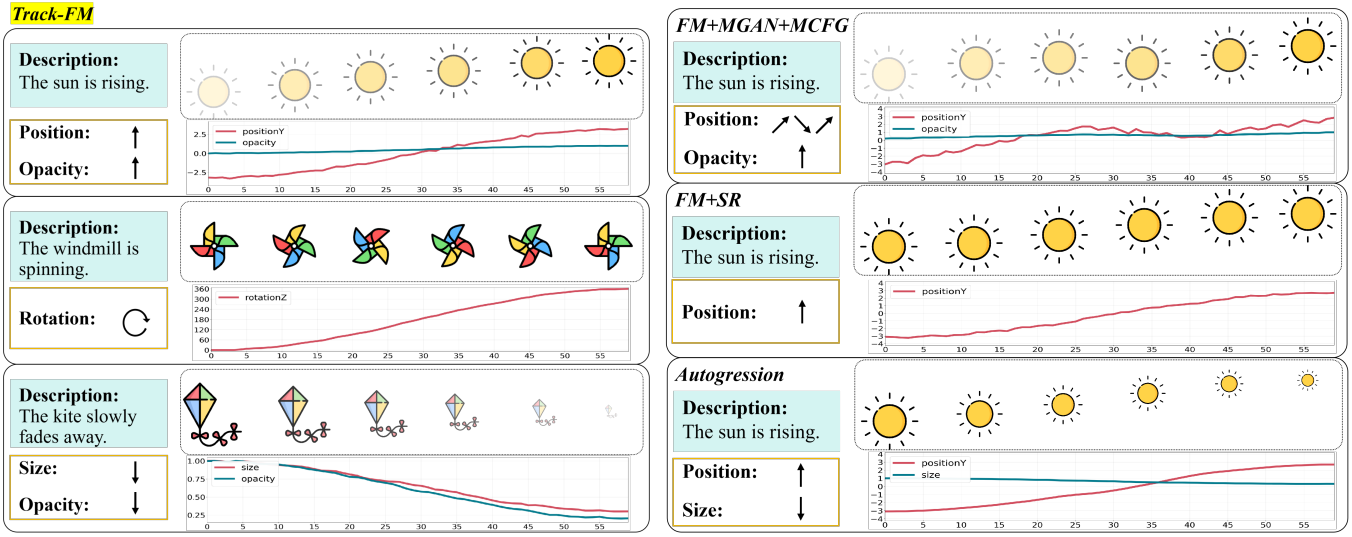


Figure 4: Rendered keyframes from generated layer-wise transformation tracks under text control. For an animation with N layers, the exported JSON file contains N such 60×10 tracks. Track-FM produces more natural and text-aligned motion with stronger global consistency (lower DTW), competitive local smoothness, and the best overall trade-off.

compliant/fully compliant); LLM score uses the same rubric on structured natural-language descriptions transcribed from the transformation tracks, rather than on the raw numerical matrices. Inter-rater agreement is high (Krippendorff's $\alpha = 0.81$, ICC= 0.84), supporting score reliability; CS/PCC/DTW provide complementary views of fidelity.

Temporal smoothness. It is measured by average total variation (ATV) and second-order difference (Δ^2):

$$\mathrm{ATV} = \frac{1}{T-1}\sum_{t=1}^{T-1}\|\hat{\mathbf{s}}_{t+1} - \hat{\mathbf{s}}_t\|_2, \tag{13}$$

$$\Delta^2 = \frac{1}{T-2}\sum_{t=2}^{T-1}\|\hat{\mathbf{s}}_{t+1} - 2\hat{\mathbf{s}}_t + \hat{\mathbf{s}}_{t-1}\|_2. \tag{14}$$

where $\hat{\mathbf{s}}_t \in \mathbb{R}^{10}$ denotes the generated transform vector at frame t and $T = 60$.

Diversity. We measure pairwise ℓ_2 distance and frame-wise variance (FWV) across multiple samples under the same prompt.

Reliability and efficiency. We measure success rate, average generation time, and average output JSON size; these metrics provide a practical proxy for pipeline usability, rather than a direct human study of downstream editing effort. Success is counted only when all checks pass (valid JSON/schema with complete finite fields, legal value ranges and frame structure, and exception-free renderer execution within timeout); otherwise the sample is marked as failure.

5.2 Experimental Results

Table 3 summarizes results in two aligned blocks: cross-family comparison and adapted OmniLottie comparison; Table 4 isolates the contribution of Track-FM components. Together, they show external competitiveness and the effect of individual Track-FM variants.

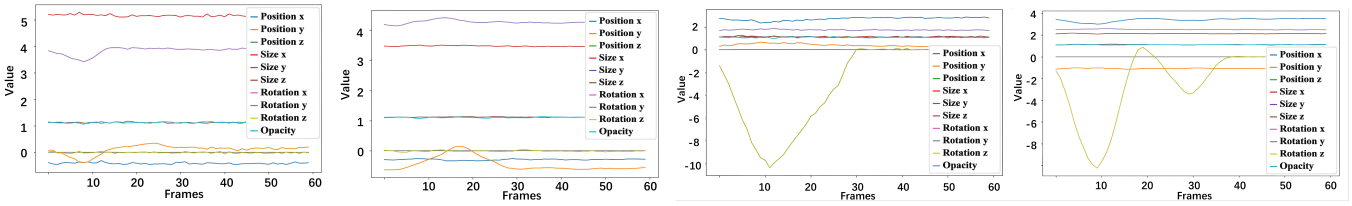
Overall Results. Across the main metrics, Track-FM achieves the strongest overall balance on reliability, alignment, long-horizon consistency, and output compactness. In the cross-family block of Table 3, Track-FM reaches the highest success rate (98.5) and the lowest DTW (2.460), while improving human/LLM alignment over matched baselines. The autoregressive baseline remains lower on ATV and Δ^2, reflecting stronger local smoothness under purely sequential generation, whereas the matched DDPM baseline shows weaker alignment and larger output JSONs. Overall, Track-FM provides the best trade-off under matched controls: compared with DDPM + MGAN + MCFG + SR, Track-FM improves success rate from 96.7 to 98.5, reduces DTW from 2.710 to 2.460, and produces smaller JSON outputs (44.7 KB versus 51.8 KB), while still improving both human and LLM scores. These gains are consistent with the modeling objective: predicting the full trajectory jointly benefits long-horizon coherence more than optimizing local continuity alone.

OmniLottie Comparison. In the adapted OmniLottie block of Table 3, performance improves steadily from OmniLottie-Text to OmniLottie-Full. Under the adapted protocol, both approaches achieve strong results; Track-FM obtains higher success rate, text alignment, and DTW, while OmniLottie-Text is slightly faster and OmniLottie-Full retains a small FWV advantage. The two approaches occupy complementary design points—program-level generation vs. per-layer track modeling—with the choice depending on whether deployment prioritizes end-to-end program synthesis or fine-grained layer-wise editability.

Visual Results. Figure 4 presents rendered keyframes from generated transformation tracks for qualitative inspection. Compared

Table 3: Main results under aligned protocols. Arrows indicate whether higher or lower is better.

Method	Reliability / Efficiency			Text / Track Alignment					Temporal Quality		Diversity
	Success ↑	Time(s) ↓	JSON(KB) ↓	User ↑	LLM ↑	CS ↑	PCC ↑	DTW ↓	ATV ↓	Δ^2 ↓	FWV ↑
Cross-family comparison											
Autoregressive	93.1	0.73	47.9	0.474	0.414	0.839	0.012	5.480	**3.560**	**0.312**	0.336
DDPM + MGAN + MCFG + SR	96.7	0.56	51.8	0.836	0.754	0.924	0.168	2.710	4.220	0.369	**0.348**
Track-FM (full)	**98.5**	0.44	**44.7**	**0.846**	**0.761**	**0.930**	**0.174**	**2.460**	3.880	0.325	0.332
Adapted OmniLottie comparison											
OmniLottie-Text (adapted)	95.6	**0.39**	**43.8**	0.786	0.721	0.902	0.139	3.020	4.290	0.389	0.258
OmniLottie-Text+Image (adapted)	96.5	0.43	45.1	0.811	0.736	0.913	0.153	2.850	4.190	0.366	0.289
OmniLottie-Full (adapted)	97.1	0.47	47.3	0.838	0.754	0.924	0.166	2.660	3.920	0.337	**0.341**
Ours (Track-FM)	**98.5**	0.44	44.7	**0.846**	**0.761**	**0.930**	**0.174**	**2.460**	**3.880**	**0.325**	0.332



(a) Case A (w/o SR) **(b) Case A (+SR)** **(c) Case B (w/o SR)** **(d) Case B (+SR)**

Figure 5: Trajectory-value comparison over 60 frames for 10 transform dimensions, without and with SR. SR reduces abrupt fluctuations, consistent with the improvements on ATV and Δ^2.

Table 4: Component ablations of Track-FM.

Variant	Reliability	Alignment		Temporal Quality		
	Success ↑	User ↑	LLM ↑	DTW ↓	ATV ↓	Δ^2 ↓
Track-FM^1 (FTM only)	95.9	0.792	0.718	2.950	4.410	0.421
Track-FM^2 (FTM+MGAN+MCFG)	97.0	0.826	0.744	2.780	4.330	0.401
Track-FM^3 (FTM+SR)	97.8	0.838	0.753	2.620	3.960	0.332
Track-FM (full)	**98.5**	**0.846**	**0.761**	**2.460**	**3.880**	**0.325**

with autoregressive and DDPM-style baselines, Track-FM follows textual motion descriptions with stronger global consistency and more coherent long-horizon trends, consistent with the DTW gains in Table 3. Across cyclic and coupled prompts, Track-FM maintains more consistent start-end behavior and fewer cross-attribute drifts. The visual comparison confirms that the measured gains remain visible after track export and playback, rather than appearing only at the parameter level.

5.3 Ablation Study

Table 4 reports component ablations on the main modules of Track-FM, using Track-FM^1 (FTM only) as the reference. Track-FM^2 and Track-FM^3 should be read as sibling branches from Track-FM^1, not as a sequential cumulative chain. Relative to Track-FM^1, adding MGAN and MCFG (Track-FM^2) primarily improves conditional faithfulness, as reflected in stronger alignment and lower DTW. Separately, adding SR on top of Track-FM^1 (Track-FM^3) improves temporal stability by reducing ATV and Δ^2 while keeping alignment competitive. Together with Figure 5, these results show that multimodal conditioning and sequence regularization provide complementary gains, with FTM as the generation backbone, MGAN and MCFG strengthening control, and SR regularizing toward smoother animations. Combining them yields the best trade-off, since the task must satisfy semantic control and animation usability simultaneously. Additional objective-level and parameter-level sensitivity analyses are provided in the Supplementary material.

6 Conclusion

In this paper, we studied text-controlled layered animation with editable transformation tracks, focusing on lightweight and directly editable motion control for layered assets. We introduced TextAE, a benchmark built from real animation projects with expert-designed layer-wise transformation tracks and a text-annotated subset, preserving layered assets and structured motion parameters.

We also presented Track-FM, a flow-matching baseline for multimodal transformation-track generation, which achieves strong alignment, reliability, and global trajectory consistency. Together, TextAE and Track-FM provide a concrete starting point for structured motion generation in real animation workflows, where tool compatibility and downstream refinement matter alongside rendered appearance. The current benchmark deliberately focuses on single-layer affine transformations to support rigorous evaluation. Extending the representation to path deformation, masking, and cross-layer coordination is a natural direction for future work.

Acknowledgments

This work was supported by Ant Group through the Ant Group Research Fund (Grant 20242452).

References

[1] Alexandre Alahi, Kratarth Goel, Vignesh Ramanathan, Alexandre Robicquet, Li Fei-Fei, and Silvio Savarese. 2016. Social lstm: Human trajectory prediction in crowded spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 961–971.

[2] Pierfrancesco Ardino, Marco De Nadai, Bruno Lepri, Elisa Ricci, and Stéphane Lathuilière. 2021. Click to move: Controlling video generation with sparse motion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 14749–14758.

[3] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, and Radu Timofte. 2020. Deepsvg: A hierarchical generative network for vector graphics animation. *Advances in Neural Information Processing Systems* 33 (2020), 16351–16361.

[4] Changgu Chen, Junwei Shu, Gaoqi He, Changbo Wang, and Yang Li. 2024. Motion-zero: Zero-shot moving object control framework for diffusion-based video generation. *arXiv preprint arXiv:2401.10150* (2024).

[5] Haoxin Chen, Menghan Xia, Yingqing He, Yong Zhang, Xiaodong Cun, Shaoshu Yang, Jinbo Xing, Yaofang Liu, Qifeng Chen, Xintao Wang, et al. 2023. Videocrafter1: Open diffusion models for high-quality video generation. *arXiv preprint arXiv:2310.19512* (2023).

[6] Prafulla Dhariwal and Alexander Nichol. 2021. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems* 34 (2021), 8780–8794.

[7] Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, et al. 2021. Cogview: Mastering text-to-image generation via transformers. *Advances in neural information processing systems* 34 (2021), 19822–19835.

[8] Oran Gafni, Adam Polyak, Oron Ashual, Shelly Sheynin, Devi Parikh, and Yaniv Taigman. 2022. Make-a-scene: Scene-based text-to-image generation with human priors. In *European conference on computer vision*. Springer, 89–106.

[9] Yu Gao, Jiancheng Huang, Xiaopeng Sun, Zequn Jie, Yujie Zhong, and Lin Ma. 2024. Matten: Video generation with mamba-attention. *arXiv preprint arXiv:2405.03025* (2024).

[10] Jiatao Gu, Ying Shen, Shuangfei Zhai, Yizhe Zhang, Navdeep Jaitly, and Josh Susskind. 2024. Kaleido diffusion: Improving conditional diffusion models with autoregressive latent modeling. *Advances in neural information processing systems* 37 (2024), 5498–5527.

[11] Shuyang Gu, Dong Chen, Jianmin Bao, Fang Wen, Bo Zhang, Dongdong Chen, Lu Yuan, and Baining Guo. 2022. Vector quantized diffusion model for text-to-image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10696–10706.

[12] Yuwei Guo, Ceyuan Yang, Anyi Rao, Zhengyang Liang, Yaohui Wang, Yu Qiao, Maneesh Agrawala, Dahua Lin, and Bo Dai. 2023. Animatediff: Animate your personalized text-to-image diffusion models without specific tuning. *arXiv preprint arXiv:2307.04725* (2023).

[13] Zekun Hao, Xun Huang, and Serge Belongie. 2018. Controllable video generation with sparse trajectories. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 7854–7863.

[14] Yingqing He, Tianyu Yang, Yong Zhang, Ying Shan, and Qifeng Chen. 2022. Latent video diffusion models for high-fidelity long video generation. *arXiv preprint arXiv:2211.13221* (2022).

[15] Jonathan Ho and Tim Salimans. 2022. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598* (2022).

[16] Jiancheng Huang, Mingfu Yan, Songyan Chen, Yi Huang, and Shifeng Chen. 2024. Magicfight: Personalized martial arts combat video generation. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 10833–10842.

[17] Jiancheng Huang, Gengwei Zhang, Zequn Jie, Siyu Jiao, Yinlong Qian, Ling Chen, Yunchao Wei, and Lin Ma. 2025. M4v: Multi-modal mamba for text-to-video generation. *arXiv preprint arXiv:2506.10915* (2025).

[18] Levon Khachatryan, Andranik Movsisyan, Vahram Tadevosyan, Roberto Henschel, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. 2023. Text2video-zero: Text-to-image diffusion models are zero-shot video generators. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 15954–15964.

[19] Quanhao Li, Zhen Xing, Rui Wang, Hui Zhang, Qi Dai, and Zuxuan Wu. 2025. Magicmotion: Controllable video generation with dense-to-sparse trajectory guidance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 12112–12123.

[20] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. 2020. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)* 39, 6 (2020), 1–15.

[21] Wei Li, Xue Xu, Xinyan Xiao, Jiachen Liu, Hu Yang, Guohao Li, Zhanpeng Wang, Zhifan Feng, Qiaoqiao She, Yajuan Lyu, et al. 2022. Upainting: Unified text-to-image diffusion generation with cross-modal guidance. *arXiv preprint arXiv:2210.16031* (2022).

[22] Xinyao Liao, Xianfang Zeng, Liao Wang, Gang Yu, Guosheng Lin, and Chi Zhang. 2025. Motionagent: fine-grained controllable video generation via motion field agent. *arXiv preprint arXiv:2502.03207* (2025).

[23] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. 2022. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747* (2022).

[24] Xingchao Liu, Chengyue Gong, and Qiang Liu. 2022. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003* (2022).

[25] Raphael Gontijo Lopes, David Ha, Douglas Eck, and Jonathon Shlens. 2019. A learned representation for scalable vector graphics. In *Proceedings of the IEEE/CVF international conference on computer vision*. 7930–7939.

[26] Wan-Duo Kurt Ma, John P Lewis, and W Bastiaan Kleijn. 2024. Trailblazer: Trajectory control for diffusion-based video generation. In *SIGGRAPH Asia 2024 Conference Papers*. 1–11.

[27] Aniruddha Mahapatra, Aliaksandr Siarohin, Hsin-Ying Lee, Sergey Tulyakov, and Jun-Yan Zhu. 2023. Text-guided synthesis of eulerian cinemagraphs. *ACM Transactions on Graphics (TOG)* 42, 6 (2023), 1–13.

[28] Yihao Meng, Hao Ouyang, Hanlin Wang, Qiuyu Wang, Wen Wang, Ka Leong Cheng, Zhiheng Liu, Yujun Shen, and Huamin Qu. 2025. Anidoc: Animation creation made easier. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18187–18197.

[29] Ron Mokady, Amir Hertz, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. 2023. Null-text inversion for editing real images using guided diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 6038–6047.

[30] Koichi Namekata, Sherwin Bahmani, Ziyi Wu, Yash Kant, Igor Gilitschenski, and David B Lindell. 2024. Sg-i2v: Self-guided trajectory control in image-to-video generation. *arXiv preprint arXiv:2411.04989* (2024).

[31] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*. 4195–4205.

[32] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. 2023. Sdxl: Improving latent diffusion models for high-resolution image synthesis. *arXiv preprint arXiv:2307.01952* (2023).

[33] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125* 1, 2 (2022), 3.

[34] Pradyumna Reddy, Michael Gharbi, Michal Lukac, and Niloy J Mitra. 2021. Im2vec: Synthesizing vector graphics without vector supervision. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 7342–7351.

[35] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10684–10695.

[36] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. 2023. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 22500–22510.

[37] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L Denton, Kamyar Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, et al. 2022. Photorealistic text-to-image diffusion models with deep language understanding. *Advances in neural information processing systems* 35 (2022), 36479–36494.

[38] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. 2020. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In *European conference on computer vision*. Springer, 683–700.

[39] Team Seedance, De Chen, Liyang Chen, Xin Chen, Ying Chen, Zhuo Chen, Zhuowei Chen, Feng Cheng, Tianheng Cheng, Yufeng Cheng, et al. 2026. Seedance 2.0: Advancing video generation for world complexity. *arXiv preprint arXiv:2604.14148* (2026).

[40] Xiaoyu Shi, Zhaoyang Huang, Fu-Yun Wang, Weikang Bian, Dasong Li, Yi Zhang, Manyuan Zhang, Ka Chun Cheung, Simon See, Hongwei Qin, et al. 2024. Motion-i2v: Consistent and controllable image-to-video generation with explicit motion modeling. In *ACM SIGGRAPH 2024 Conference Papers*. 1–11.

[41] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, et al. 2022. Make-a-video: Text-to-video generation without text-video data. *arXiv preprint arXiv:2209.14792* (2022).

[42] Guy Tevet, Sigal Raab, Brian Gordon, Yonatan Shafir, Daniel Cohen-Or, and Amit H Bermano. 2022. Human motion diffusion model. *arXiv preprint arXiv:2209.14916* (2022).

[43] Ruben Villegas, Mohammad Babaeizadeh, Pieter-Jan Kindermans, Hernan Moraldo, Han Zhang, Mohammad Taghi Saffar, Santiago Castro, Julius Kunze, and Dumitru Erhan. 2022. Phenaki: Variable length video generation from open

domain textual description. *arXiv preprint arXiv:2210.02399* (2022).

[44] Jiuniu Wang, Hangjie Yuan, Dayou Chen, Yingya Zhang, Xiang Wang, and Shiwei Zhang. 2023. Modelscope text-to-video technical report. *arXiv preprint arXiv:2308.06571* (2023).

[45] Zhouxia Wang, Ziyang Yuan, Xintao Wang, Yaowei Li, Tianshui Chen, Menghan Xia, Ping Luo, and Ying Shan. 2024. Motionctrl: A unified and flexible motion controller for video generation. In *ACM SIGGRAPH 2024 Conference Papers*. 1–11.

[46] Weijia Wu, Zhuang Li, Yuchao Gu, Rui Zhao, Yefei He, David Junhao Zhang, Mike Zheng Shou, Yan Li, Tingting Gao, and Di Zhang. 2024. Draganything: Motion control for anything using entity representation. In *European Conference on Computer Vision*. Springer, 331–348.

[47] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, and Qian Yu. 2024. Svgdreamer: Text guided svg generation with diffusion model. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 4546–4555.

[48] Yiying Yang, Wei Cheng, Sijin Chen, Honghao Fu, Xianfang Zeng, Yujun Cai, Gang Yu, and Xingjun Ma. 2026. OmniLottie: Generating Vector Animations via Parameterized Lottie Tokens. *arXiv preprint arXiv:2603.02138* (2026).

[49] Guy Yariv, Yuval Kirstain, Amit Zohar, Shelly Sheynin, Yaniv Taigman, Yossi Adi, Sagie Benaim, and Adam Polyak. 2025. Through-the-mask: Mask-based motion trajectories for image-to-video generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18198–18208.

[50] Shengming Yin, Chenfei Wu, Jian Liang, Jie Shi, Houqiang Li, Gong Ming, and Nan Duan. 2023. Dragnuwa: Fine-grained control in video generation by integrating text, image, and trajectory. *arXiv preprint arXiv:2308.08089* (2023).

[51] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2023. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF international conference on computer vision*. 3836–3847.

[52] Mingyuan Zhang, Zhongang Cai, Liang Pan, Fangzhou Hong, Xinying Guo, Lei Yang, and Ziwei Liu. 2024. Motiondiffuse: Text-driven human motion generation with diffusion model. *IEEE transactions on pattern analysis and machine intelligence* 46, 6 (2024), 4115–4128.

[53] Zhenghao Zhang, Junchao Liao, Menghao Li, Zuozhuo Dai, Bingxue Qiu, Siyu Zhu, Long Qin, and Weizhi Wang. 2025. Tora: Trajectory-oriented diffusion transformer for video generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2063–2073.

[54] Zhongwei Zhang, Fuchen Long, Zhaofan Qiu, Yingwei Pan, Wu Liu, Ting Yao, and Tao Mei. 2025. Motionpro: A precise motion controller for image-to-video generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 27957–27967.

[55] Kaifeng Zhao, Gen Li, and Siyu Tang. 2024. DartControl: A diffusion-based autoregressive motion model for real-time text-driven motion control. *arXiv preprint arXiv:2410.05260* (2024).